

## Practical-8

### Aim:

To analyze stock market data using **Apache Spark and PySpark** by grouping stock records based on stock symbols and calculating average, maximum, and minimum stock prices to identify performance trends.

### Problem Definition:

You are a Data Analytics Engineer at FinEdge Securities Ltd., a financial services company that provides stock market insights to investors. The company continuously receives stock price updates from multiple exchanges. Investors want to understand stock performance trends and identify the best investment opportunities. Traditional data processing systems struggle to handle the continuously growing volume of financial data.

### Objectives:

1. To create a Spark Session for processing stock market data.
2. To load the stock dataset into a Spark DataFrame.
3. To display the stock records and dataset schema.
4. To group stock records based on stock symbols.
5. To calculate the average stock price for each stock.
6. To calculate the maximum and minimum stock prices.
7. To generate a stock summary report.
8. To identify stocks with higher observed average prices in the given dataset.
9. To interpret stock market trends using the analyzed data.
10. To generate analytical investment recommendations based on the available dataset.

### Objective/Output:

Tasks: Create Spark Session. Load stock dataset into Spark DataFrame. Display schema and stock records.

```

gitanshu@Parth: ~/spark_stock_practical
gitanshu@Parth:~$ cd ~
gitanshu@Parth:~$ mkdir -p spark_stock_practical
gitanshu@Parth:~$ cd spark_stock_practical
gitanshu@Parth:~/spark_stock_practical$ pwd
/home/gitanshu/spark_stock_practical
gitanshu@Parth:~/spark_stock_practical$ source ~/pyspark-env/bin/activate
(pyspark-env) gitanshu@Parth:~/spark_stock_practical$ python -c "import pyspark; print(pyspark.__version__)"
4.2.0
(pyspark-env) gitanshu@Parth:~/spark_stock_practical$ nano stocks.csv
(pyspark-env) gitanshu@Parth:~/spark_stock_practical$ cat stocks.csv
date,symbol,open,high,low,close,volume
2026-01-01,AAPL,180,185,178,183,1200000
2026-01-02,AAPL,183,188,181,187,1350000
2026-01-03,AAPL,187,190,184,189,1420000
2026-01-04,AAPL,189,192,186,191,1500000
2026-01-05,AAPL,191,195,189,194,1600000
2026-01-01,GOOGL,140,145,138,143,1100000
2026-01-02,GOOGL,143,147,141,146,1180000
2026-01-03,GOOGL,146,150,144,149,1250000
2026-01-04,GOOGL,149,152,147,151,1300000
2026-01-05,GOOGL,151,155,149,154,1380000
2026-01-01,MSFT,370,375,368,373,1000000
2026-01-02,MSFT,373,379,371,377,1080000
2026-01-03,MSFT,377,382,375,380,1150000
2026-01-04,MSFT,380,386,378,384,1220000
2026-01-05,MSFT,384,390,382,388,1300000
2026-01-01,AMZN,150,154,148,152,900000
2026-01-02,AMZN,152,156,150,155,950000
2026-01-03,AMZN,155,159,153,158,1020000
2026-01-04,AMZN,158,162,156,161,1080000
2026-01-05,AMZN,161,165,159,164,1150000

```

```

26/09/30 11:15:55 INFO DAGScheduler: Asked to cleanup jobs for query execution 2

```

| date       | symbol | open | high | low | close | volume  |
|------------|--------|------|------|-----|-------|---------|
| 2026-01-01 | AAPL   | 180  | 185  | 178 | 183   | 1200000 |
| 2026-01-02 | AAPL   | 183  | 188  | 181 | 187   | 1350000 |
| 2026-01-03 | AAPL   | 187  | 190  | 184 | 189   | 1420000 |
| 2026-01-04 | AAPL   | 189  | 192  | 186 | 191   | 1500000 |
| 2026-01-05 | AAPL   | 191  | 195  | 189 | 194   | 1600000 |
| 2026-01-01 | GOOGL  | 140  | 145  | 138 | 143   | 1100000 |
| 2026-01-02 | GOOGL  | 143  | 147  | 141 | 146   | 1180000 |
| 2026-01-03 | GOOGL  | 146  | 150  | 144 | 149   | 1250000 |
| 2026-01-04 | GOOGL  | 149  | 152  | 147 | 151   | 1300000 |
| 2026-01-05 | GOOGL  | 151  | 155  | 149 | 154   | 1380000 |
| 2026-01-01 | MSFT   | 370  | 375  | 368 | 373   | 1000000 |
| 2026-01-02 | MSFT   | 373  | 379  | 371 | 377   | 1080000 |
| 2026-01-03 | MSFT   | 377  | 382  | 375 | 380   | 1150000 |
| 2026-01-04 | MSFT   | 380  | 386  | 378 | 384   | 1220000 |
| 2026-01-05 | MSFT   | 384  | 390  | 382 | 388   | 1300000 |
| 2026-01-01 | AMZN   | 150  | 154  | 148 | 152   | 900000  |
| 2026-01-02 | AMZN   | 152  | 156  | 150 | 155   | 950000  |
| 2026-01-03 | AMZN   | 155  | 159  | 153 | 158   | 1020000 |
| 2026-01-04 | AMZN   | 158  | 162  | 156 | 161   | 1080000 |
| 2026-01-05 | AMZN   | 161  | 165  | 159 | 164   | 1150000 |

```

Dataset Schema:
root
 |-- date: date (nullable = true)
 |-- symbol: string (nullable = true)
 |-- open: integer (nullable = true)
 |-- high: integer (nullable = true)
 |-- low: integer (nullable = true)
 |-- close: integer (nullable = true)
 |-- volume: integer (nullable = true)

```

Group records based on stock symbol.

```

26/09/30 11:15:57 INFO CodeGenerator: Code generated in 7.270832 ms
26/09/30 11:15:57 INFO DAGScheduler: Asked to cleanup jobs for query execution 3

```

| symbol | count |
|--------|-------|
| AAPL   | 5     |
| GOOGL  | 5     |
| AMZN   | 5     |
| MSFT   | 5     |

Calculate average stock price.

```

26/09/30 11:15:58 INFO DAGScheduler: Job 8 finished: $anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768, took 136.833888 ms
26/09/30 11:15:58 INFO CodeGenerator: Code generated in 7.784946 ms
26/09/30 11:15:58 INFO DAGScheduler: Asked to cleanup jobs for query execution 4
+-----+
|symbol|average_price|
+-----+
| MSFT |          380.4|
| AAPL |          188.8|
| AMZN |          158.0|
| GOOGL|          148.6|
+-----+

```

Calculate maximum stock price.

```

26/09/30 11:15:58 INFO DAGScheduler: Job 9 finished: $anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768, took 86.496217 ms
26/09/30 11:15:58 INFO CodeGenerator: Code generated in 7.691738 ms
26/09/30 11:15:58 INFO DAGScheduler: Asked to cleanup jobs for query execution 5
+-----+
|symbol|maximum_price|
+-----+
| MSFT |          388|
| AAPL |          194|
| AMZN |          164|
| GOOGL|          154|
+-----+

```

Calculate minimum stock price.

```

26/09/30 11:15:59 INFO DAGScheduler: Job 10 finished: $anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768, took 127.768988 ms
26/09/30 11:15:59 INFO DAGScheduler: Asked to cleanup jobs for query execution 6
+-----+
|symbol|minimum_price|
+-----+
| GOOGL|          143|
| AMZN |          152|
| AAPL |          183|
| MSFT |          373|
+-----+

```

Generate stock summary report.

```

26/09/30 11:16:00 INFO DAGScheduler: Asked to cleanup jobs for query execution 7
+-----+
|symbol|average_price|maximum_price|minimum_price|
+-----+
|  MSFT |          380.4|          388|          373|
|  AAPL |          188.8|          194|          183|
|  AMZN |          158.0|          164|          152|
|  GOOGL|          148.6|          154|          143|
+-----+

```

Identify best-performing stocks.

```

26/09/30 11:16:00 INFO DAGScheduler: Asked to cleanup jobs for query execution 8
+-----+
|symbol|average_price|maximum_price|minimum_price|
+-----+
|  MSFT |          380.4|          388|          373|
|  AAPL |          188.8|          194|          183|
|  AMZN |          158.0|          164|          152|
|  GOOGL|          148.6|          154|          143|
+-----+

```

Interpret market trends.

```
=====
MARKET TREND INTERPRETATION
=====

The stock summary is analyzed using average, maximum and
minimum closing prices.

A stock with a higher average closing price indicates a
higher average price level during the analyzed period.

The difference between maximum and minimum price indicates
the observed price range during the period.

The results are based only on the supplied historical dataset.
They should not be treated as guaranteed future performance.
```

Generate investment recommendations.

```
Based on the dataset, MSFT has the highest average closing price of 380.4.

Analytical recommendation:
Stocks can be further evaluated using average price,
price range, trading volume, volatility and historical
price trends before making any investment decision.

This recommendation is based only on the sample dataset
and is not financial advice.

26/09/30 11:16:01 INFO SparkContext: SparkContext is stopping with exitCode 0 from stop at NativeMethodAccessorImpl.java:0
```

## Key Questions /Analysis:

Q1. Why is Apache Spark suitable for financial analytics?

Apache Spark is suitable for financial analytics because it can process **large volumes of financial data quickly** using distributed and parallel processing.

Financial systems may continuously generate data such as:

- Stock prices
- Trading volumes
- Transaction records
- Market data
- Historical prices

Spark can process this data using **DataFrames, Spark SQL, caching, partitioning, and parallel execution.**

Q2. What is the purpose of grouping stock data by stock symbol?

A stock symbol uniquely identifies a company or financial instrument.

For example:

AAPL

MSFT

GOOGL

AMZN

When we use:

```
df.groupBy("symbol")
```

Spark creates separate groups for each stock.

Example:

AAPL → All AAPL records

MSFT → All MSFT records

AMZN → All AMZN records

GOOGL → All GOOGL records

This allows us to calculate statistics separately for each company.

Q3. How does average stock price help investors?

Average stock price represents the **typical price level** of a stock during the analyzed period.

For example:

AAPL → Average closing price = ₹188.80

MSFT → Average closing price = ₹380.40

It can help compare the historical price levels of different stocks.

However, average price alone does **not** tell us whether a stock generated the highest return or was the least risky.

Q4. Why are maximum and minimum prices important?

Maximum and minimum prices show the **observed price range** during the selected period.

For example:

Stock: AAPL

Minimum = ₹183

Maximum = ₹194

This tells us that the stock's closing price varied between ₹183 and ₹194 during the analyzed period.

The price range can provide an initial indication of how much the price moved.

Q5. How can Spark support real-time stock market monitoring?

Apache Spark can support real-time monitoring through **Spark Structured Streaming**.

The basic architecture can be:

Stock Exchanges



Streaming Data



Kafka / Streaming Source



Spark Structured Streaming



Real-Time Processing



Analysis / Alerts / Dashboard

Spark can continuously process incoming stock data and calculate metrics such as:

- Current prices

- Trading volume
- Price changes
- Moving averages
- Returns
- Alerts

### Supplementary Problems:

Calculate daily stock returns.

```
26/09/30 12:30:26 INFO SparkContext: SparkContext is stopping with exitCode 0 from stop at NativeMethodAccessorImpl.java:0.
+-----+-----+-----+-----+
|date|symbol|close|previous_close|daily_return|
+-----+-----+-----+-----+
|2026-01-01|AAPL|183|NULL|NULL|
|2026-01-02|AAPL|187|183|2.19|
|2026-01-03|AAPL|189|187|1.07|
|2026-01-04|AAPL|191|189|1.06|
|2026-01-05|AAPL|194|191|1.57|
|2026-01-01|AMZN|152|NULL|NULL|
|2026-01-02|AMZN|155|152|1.97|
|2026-01-03|AMZN|158|155|1.94|
|2026-01-04|AMZN|161|158|1.9|
|2026-01-05|AMZN|164|161|1.86|
|2026-01-01|GOOGL|143|NULL|NULL|
|2026-01-02|GOOGL|146|143|2.1|
|2026-01-03|GOOGL|149|146|2.05|
|2026-01-04|GOOGL|151|149|1.34|
|2026-01-05|GOOGL|154|151|1.99|
|2026-01-01|MSFT|373|NULL|NULL|
|2026-01-02|MSFT|377|373|1.07|
|2026-01-03|MSFT|380|377|0.8|
|2026-01-04|MSFT|384|380|1.05|
|2026-01-05|MSFT|388|384|1.04|
+-----+-----+-----+-----+
26/09/30 12:30:26 INFO SparkUI: Stopped Spark web UI at http://10.255.255.254:4040
```

Identify top-performing stock over a period.

```
26/09/30 12:34:00 INFO DAGScheduler: Asked to cleanup jobs for query execution 2
+-----+-----+-----+-----+
|symbol|first_price|last_price|period_return|
+-----+-----+-----+-----+
|AMZN|152|164|7.89|
|GOOGL|143|154|7.69|
|AAPL|183|194|6.01|
|MSFT|373|388|4.02|
+-----+-----+-----+-----+
Top-Performing Stock:
26/09/30 12:34:00 INFO FileSourceStrategy: Pushed Filters:
```

Compute moving average for stock prices.

```

26/09/30 12:35:25 INFO DAGScheduler: Asked to cleanup jobs for query execution 2
+-----+-----+-----+-----+
|date|symbol|close|moving_average_3day|
+-----+-----+-----+-----+
|2026-01-01|AAPL|183|183.0|
|2026-01-02|AAPL|187|185.0|
|2026-01-03|AAPL|189|186.33|
|2026-01-04|AAPL|191|189.0|
|2026-01-05|AAPL|194|191.33|
|2026-01-01|AMZN|152|152.0|
|2026-01-02|AMZN|155|153.5|
|2026-01-03|AMZN|158|155.0|
|2026-01-04|AMZN|161|158.0|
|2026-01-05|AMZN|164|161.0|
|2026-01-01|GOOGL|143|143.0|
|2026-01-02|GOOGL|146|144.5|
|2026-01-03|GOOGL|149|146.0|
|2026-01-04|GOOGL|151|148.67|
|2026-01-05|GOOGL|154|151.33|
|2026-01-01|MSFT|373|373.0|
|2026-01-02|MSFT|377|375.0|
|2026-01-03|MSFT|380|376.67|
|2026-01-04|MSFT|384|380.33|
|2026-01-05|MSFT|388|384.0|
+-----+-----+-----+-----+

26/09/30 12:35:25 INFO SparkContext: SparkContext is stopping with exitCode 0 from stop at NativeMethodAccessorImpl.java:0.
26/09/30 12:35:26 INFO SparkContext: SparkContext is stopped.

```

Compare stock volatility across companies.

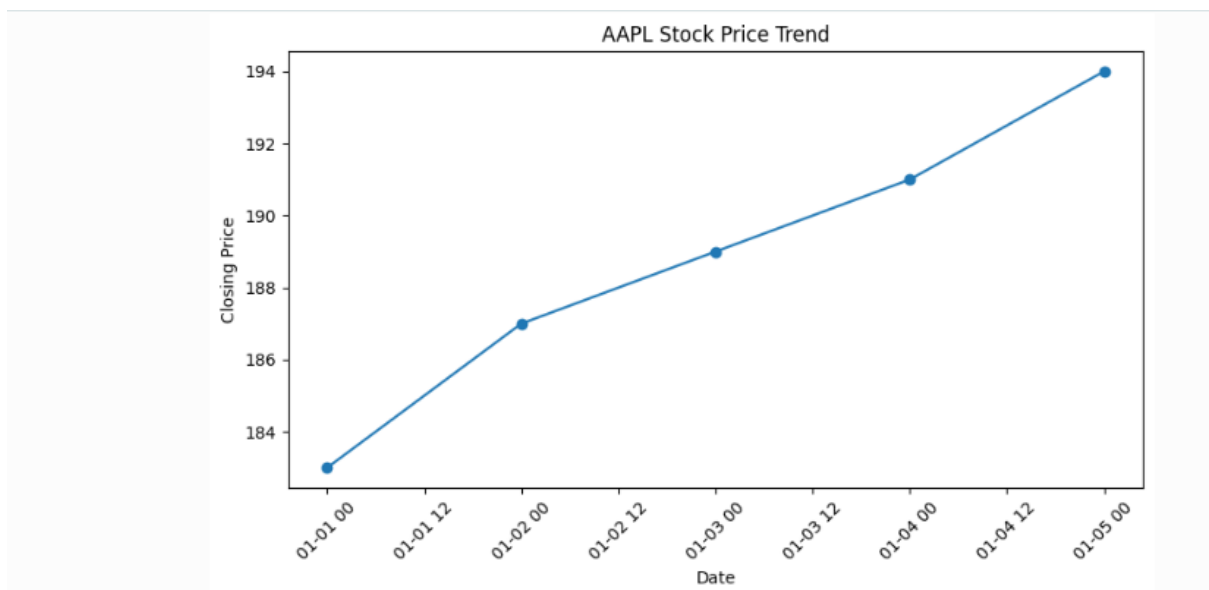
```

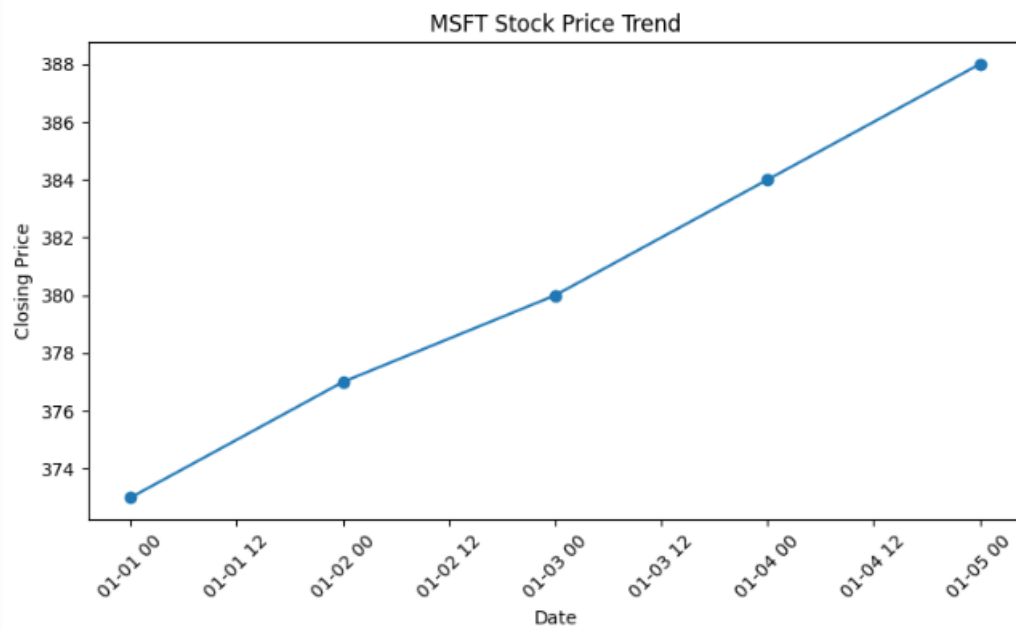
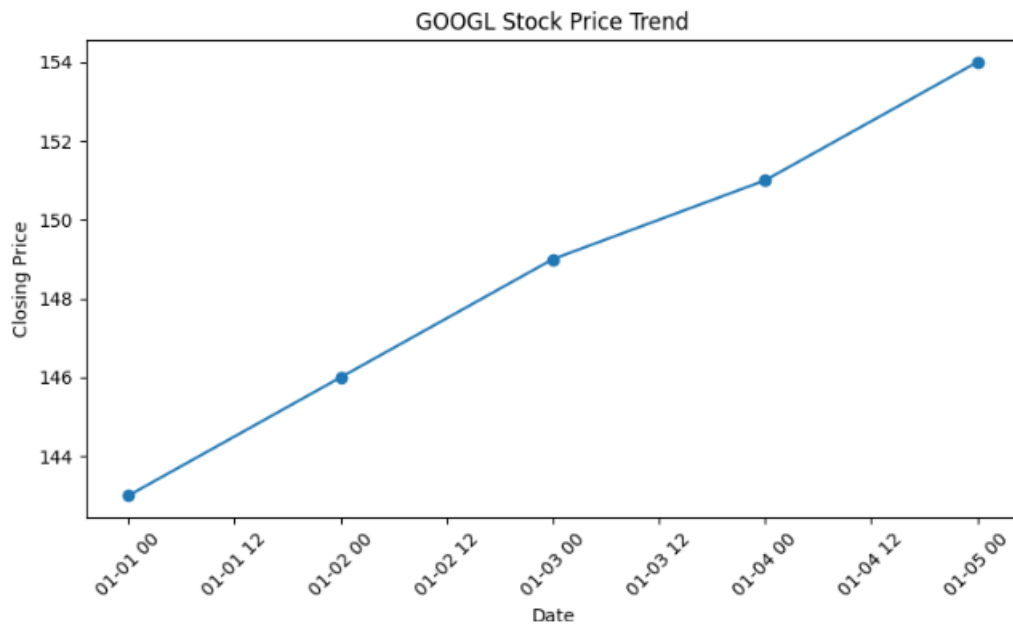
26/09/30 12:39:11 INFO SparkContext: SparkContext is stopping with exitCode 0 from stop at NativeMethodAccessorImpl.java:0.
+-----+-----+
|symbol|volatility|
+-----+-----+
|AAPL|0.53|
|GOOGL|0.36|
|MSFT|0.13|
|AMZN|0.05|
+-----+-----+

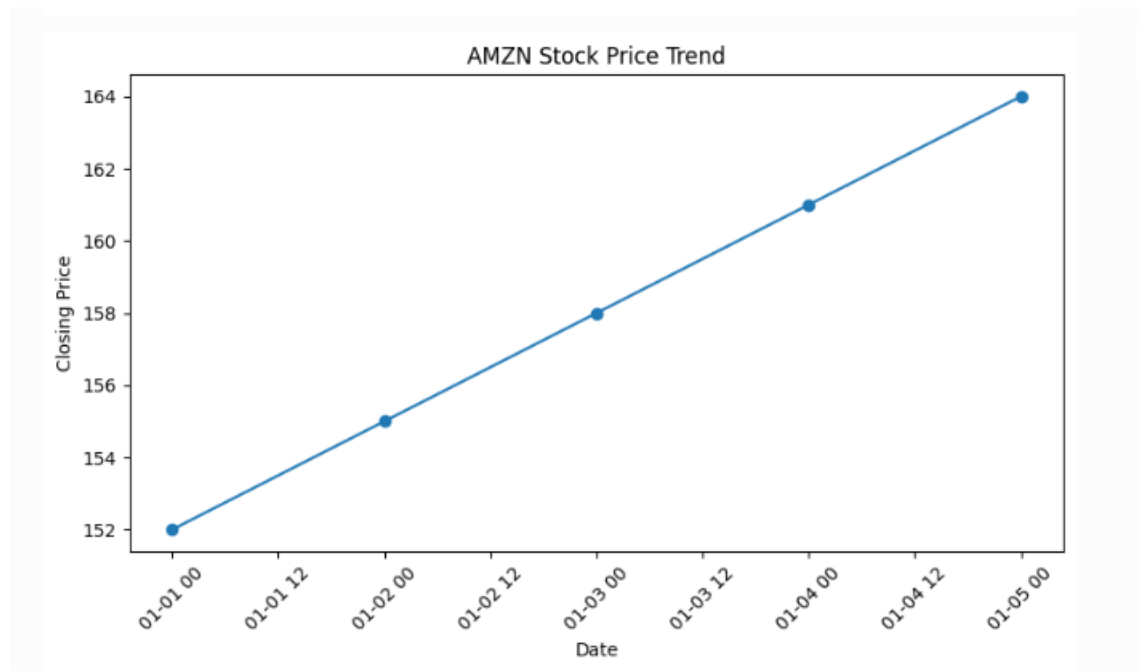
26/09/30 12:39:11 INFO SparkUI: Stopped Spark web UI at http://10.255.255.254:4040

```

Visualize stock price trends using charts.







### Tools and Technologies:

- **Programming Language:** Python 3
- **Processing Framework:** Apache Spark
- **Spark API:** PySpark
- **Data Processing:** Spark DataFrames
- **Input Data Format:** CSV
- **Data Source:** Stock Market Dataset
- **Data Analysis:** Spark SQL / DataFrame Operations
- **Grouping:** groupBy()
- **Aggregation:** agg(), avg(), max(), min()
- **Sorting:** orderBy(), desc()
- **Data Rounding:** round()
- **Operating System:** Ubuntu Linux / WSL2
- **Execution Environment:** Local Spark (local[\*])
- **Package Management:** Python pip
- **Virtual Environment:** Python venv

- **Development Environment:** Ubuntu Terminal / Nano
- **Execution Tool:** spark-submit
- **Dataset File:** stocks.csv

### **Conclusion:**

The practical successfully demonstrated the use of **PySpark for stock market data analysis**. The stock dataset was loaded into a Spark DataFrame, its schema and records were examined, and stock data was grouped according to individual symbols. Average, maximum, and minimum closing prices were calculated to generate a summary of stock performance.

The analysis helped identify price patterns and compare stocks within the given dataset. It also demonstrated how Spark DataFrame operations can efficiently process and analyze large-scale financial data. The generated recommendations are based only on the historical sample data and should be considered **analytical observations rather than actual financial advice**.